Workshop Module: Simulating Vehicular Communication with Mininet-WiFi and SUMO

Objective

The goal of this workshop is to simulate a vehicular network using Mininet-WiFi and SUMO. The focus will be on sending ping floods between vehicles to understand network communication and performance.

Prerequisites

- Basic knowledge of Python programming.
- Familiarity with networking concepts.
- Mininet-WiFi and SUMO installed on your system.

Script Overview

The script sets up a simulated vehicular network using Mininet-WiFi and SUMO, then sends ping floods between cars to simulate communication.

Detailed Script Explanation

1. Importing Modules and Environment Setup

```
import sys
import os
import math
import subprocess
from threading import Thread

# Ensure SUMO_HOME is set
if 'SUMO_HOME' not in os.environ:
    os.environ['SUMO_HOME'] = "/usr/share/sumo"

sys.path.append(os.path.join(os.environ['SUMO_HOME'], 'tools'))

import traci
from mininet.log import setLogLevel, info
from mn_wifi.cli import CLI
from mn_wifi.net import Mininet_wifi
from mn_wifi.link import wmediumd, mesh
from mn_wifi.wmediumdConnector import interference
```

Explanation:

- **Importing Modules:**
  - **Standard Libraries:** `sys`, `os`, `math`, and `subprocess` are used for system operations and math calculations.

    - **Threading:** `Thread` is imported from `threading` for executing tasks concurrently.
- **Environment Setup:**
    - The script checks if `SUMO_HOME` is set in the environment variables. If not, it sets it to the default path.
    - The SUMO tools directory is appended to the Python path, allowing access to SUMO's Python libraries.
- **Additional Imports:**
    - **`traci`:** Used for interfacing with SUMO to simulate and control traffic scenarios.
    - **`Mininet-WiFi` Libraries:** Used to create and manage the wireless network simulation.

2. Utility Functions

Calculating Subnet Mask:

```
def calculate_subnet_mask(number_of_nodes):
    number_of_ips_needed = number_of_nodes + 2
    bits_needed_for_hosts = math.ceil(math.log2(number_of_ips_needed))
    subnet_mask = 32 - bits_needed_for_hosts
    return subnet_mask
```

- **Purpose:**
    - This function calculates the subnet mask needed for a given number of nodes (cars).
    - Ensures each car has a unique IP address and fits within the subnet.
- **How It Works:**
    - Calculates the number of IPs needed, then computes the number of host bits required.
    - Converts the number of host bits into a subnet mask.

Starting and Stopping Traffic Capture:

```
def start_tcpdump(pcap_file):
    cmd = ["tcpdump", "-i", "any", "-w", pcap_file]
    info(f"Starting tcpdump with output file {pcap_file}\n")
    process = subprocess.Popen(cmd, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
    return process

def stop_tcpdump(process):
    if process:
        process.terminate()
```

- **Purpose:**
    - These functions manage `tcpdump` to capture network traffic during the simulation.
- **How They Work:**

- **start_tcpdump:** Starts a `tcpdump` process to capture all network traffic and writes it to a `.pcap` file.
- **stop_tcpdump:** Stops the `tcpdump` process, terminating the capture.

2. **Ping Flooding Logic**

```
def ping_flood(source, target_ip):
    cmd = ["ping", "-i 0.01", target_ip]
    result = subprocess.run(cmd, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
    if result.returncode == 0:
        info(f"Successfully sent ping flood from {source.name} to
{target_ip}")
```

- **Purpose:**
  - Sends a flood of ping requests to a target IP address.
- **How It Works:**
  - Uses the `ping` command to send requests with a high frequency (`-i 0.01` specifies an interval of 10ms between packets).
  - Executes the command using `subprocess.run`, suppressing output and checking the return code to confirm success.

3. **Executing Flood on All Car**

```
def flood_all(net):
    def flood_car(car1, car2):
        target_ip = car2.params['ip']
        ping_flood(car1, target_ip)

    threads = []
    for car1 in net.cars:
        for car2 in net.cars:
            if car1 != car2:
                thread = Thread(target=flood_car, args=(car1,
car2))
                threads.append(thread)
                thread.start()

    for thread in threads:
        thread.join()
```

- **Purpose:**
  - This function sends ping floods from each car to every other car in the network.
- **How It Works:**
  - **flood_car:** Helper function that calls `ping_flood` for a pair of cars.
  - Uses threading to initiate ping floods in parallel, simulating simultaneous network activity and creating realistic network congestion.
  - The function iterates over each pair of cars and starts a thread for each flood operation.
  - After starting all threads, it waits for each thread to complete.

4. Network Topology Setup and Execution

```
def topology(num_cars, sumo_config_file, duration):
    net = Mininet_wifi(link=wmediumd, wmediumd_mode=interference)

    info("*** Creating nodes\n")
    for id in range(num_cars):
        car = net.addCar(f'car{id + 1}', wlans=1, encrypt=['wpa2', ''])
# One interface per car

    info("*** Configuring Propagation Model\n")
    net.setPropagationModel(model="logDistance", exp=2.8)

    info("*** Configuring nodes\n")
    net.configureNodes()
    for car in net.cars:
        net.addLink(car, intf=car.wintfs[0].name, cls=mesh, ssid='mesh-
ssid', channel=5, ht_cap='HT40+')

    info("*** Starting network\n")
    net.build()

    subnet_mask = calculate_subnet_mask(num_cars)
    base_ip = 192 * 256**3 + 168 * 256**2

    for id, car in enumerate(net.cars):
        ip = base_ip + id + 1
        ip_str = f'{(ip // (256**3)) % 256}.{(ip // (256**2)) % 256}.{(ip
// 256) % 256}.{ip % 256}'
        car.setIP(f'{ip_str}/{subnet_mask}',
intf=f'{car.wintfs[0].name}')
        info(f"Car {car.name} assigned IP {ip_str}/{subnet_mask}\n")

    info("*** Running simulation\n")

    sumoCmd = ["sumo-gui", "-c", sumo_config_file, "--start", "--delay",
"500"]
    sumo_process = subprocess.Popen(sumoCmd)
    time.sleep(2)  # Give SUMO time to start
    traci.start(["sumo", "-c", sumo_config_file], port=8813)
    traci.setOrder(1)

    pcap_file = f"{os.getcwd()}/traffic.pcap"
    tcpdump_process = start_tcpdump(pcap_file)

    start_time = time.time()
    while time.time() - start_time < duration:
        traci.simulationStep()
        flood_all(net)

    stop_tcpdump(tcpdump_process)
    traci.close()
    net.stop()
    subprocess.run(["mn", "-c"])
    sys.exit(0)
```

- **Purpose:**

    - Sets up and runs the network simulation, integrating Mininet-WiFi and SUMO.
- **How It Works:**
    - **Node Creation:** Uses Mininet-WiFi to create car nodes, each with one wireless interface.
    - **Propagation Model:** Configures the network with a log-distance propagation model, simulating realistic wireless communication.
    - **Link Configuration:** Establishes mesh links between nodes to simulate ad-hoc vehicular communication.
    - **IP Assignment:** Calculates and assigns unique IP addresses to each car in the network.
    - **SUMO Integration:** Launches SUMO-GUI with the specified configuration and starts the Traci interface for control during simulation.
    - **Traffic Capture:** Initiates `tcpdump` to record all network traffic.
    - **Simulation Execution:** Continuously steps through the SUMO simulation, running the flood logic until the specified duration elapses.
    - **Cleanup:** Stops network processes and clears Mininet states.

5. **Main Execution Block**

```
if __name__ == '__main__':
    setLogLevel('info')
    num_cars = 100  # number of cars
    duration = 60  # Duration of the simulation in seconds
    sumo_config_file = '6lane.sumocfg'  # SUMO configuration file
    topology(num_cars, sumo_config_file, duration)
```

- **Purpose:**
    - Executes the simulation setup and runs the topology defined earlier.
- **How It Works:**
    - **Log Level:** Sets the logging level to 'info' for detailed output during execution.
    - **Parameters:**
        - `num_cars`: Number of cars to simulate.
        - `duration`: Length of the simulation in seconds.
        - `sumo_config_file`: Specifies the SUMO configuration file to use.
    - **Run Topology:** Calls the `topology` function with specified parameters to start the simulation.